INF384 · TEMAS AVANZADOS EN INGENIERÍA DE SOFTWARE 1 · 2026-2

# Sesión 08

## Automatización de calidad: contratos, end to end y orquestación

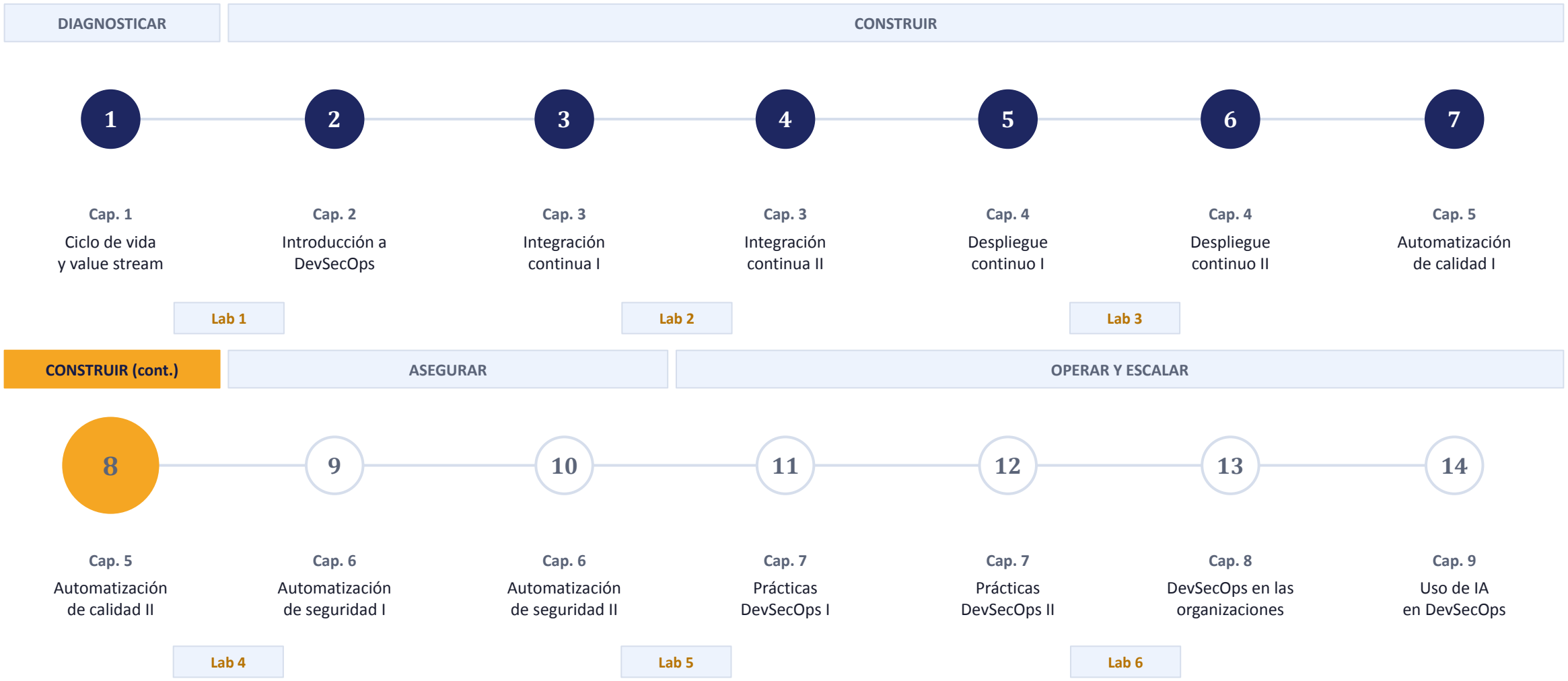
*Pruebas entre servicios, recorridos de usuario y la secuencia de pruebas a lo largo del pipeline*

---

Capítulo 5 (2 de 2) · 3 horas | RA2 · RA4 | Prof. José Felipe del Castillo Salazar

INF384 · 9 CAPÍTULOS · 14 SESIONES · 42 HORAS

# La ruta del curso



SESIÓN 8

# Objetivo de la sesión y ruta

1. Pruebas end to end y Playwright
2. Pruebas sobre el ambiente desplegado: humo y rendimiento
3. Orquestación de pruebas en el pipeline
4. Preparación para Examen

**Resultados de aprendizaje vinculados: RA2 y RA4 — calidad del ciclo de vida y resiliencia de lo desplegado.**

CONTINUIDAD CON LA SESIÓN 7

# Dónde estamos en el pipeline

| Commit | Build | **Pruebas** | Calidad | Seguridad | Publicar | **Desplegar** | **Observar** |
|---|---|---|---|---|---|---|---|

## Que ya vimos de Calidad

- La proporción de pruebas se decide por el costo de detectar cada defecto.
- Las unitarias funcionan como compuerta si son rápidas, aisladas y repetibles.
- La integración acotada corre en el runner con dependencias efímeras.

## ¿Qué falta ver?

- ¿Cómo se comprueba que dos servicios siguen siendo compatibles si se despliegan por separado?
- ¿Qué flujos de usuario justifican una prueba end to end?
- ¿Qué pruebas corren contra el ambiente desplegado?
- ¿Qué resultado de prueba detiene el despliegue a producción?

**La ejecución de pruebas se distribuyen entre etapa de pruebas, despliegue y post despliegue en la observabilidad.**

1

# Pruebas end to end

Los recorridos de usuario que justifican el nivel más caro de la pirámide

# Qué recorridos merecen una prueba end to end

1. **Recorridos que generan ingreso o cumplen una obligación**
   Pagar, emitir un comprobante, agendar una cita, registrar un despacho.

2. **Recorridos que cruzan varios servicios**
   Ninguna prueba inferior los recorre completos: cada servicio solo prueba su tramo.

3. **Recorridos con incidentes previos en producción**
   El postmortem del incidente deja una prueba end to end como acción concreta.

4. **Un conjunto acotado y revisado periódicamente**
   Cada recorrido nuevo reemplaza a otro o justifica su costo en minutos de pipeline.

**Cada recorrido end to end cuesta minutos en cada ejecución y horas de mantenimiento al año. Entra a la suite cuando su falla en producción costaría más que eso.**

# Anatomía de una prueba end to end confiable

| Aspecto | Frágil | Confiable |
|---|---|---|
| Selección | Por posición o por texto visible | Por atributos estables dedicados a la prueba |
| Espera | Fija: sleep de cinco segundos | Por condición: elemento visible, respuesta 200 |
| Datos | Cargados a mano en el ambiente | Creados por la prueba vía API y eliminados al final |
| Independencia | Encadenadas: la 2 usa lo que creó la 1 | Cada prueba corre sola y en cualquier orden |
| Verificación | Toda la pantalla, píxel a píxel | El resultado de negocio del recorrido |

Las cinco propiedades de la sesión 7 aplican también aquí. La end to end las pierde con más facilidad porque depende de un ambiente completo.

# End to end sobre una API serverless

TEST/E2E/PEDIDO.E2E.JS

```
it("registra el pedido y devuelve la tarifa", async () => {
  const url = process.env.URL_API;                    // 1
  const ref = `e2e-${Date.now()}`;                    // 2

  const res = await fetch(`${url}/pedidos`, {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify({ pesoKg: 10, urgente: true, ref }),
  });

  expect(res.status).toBe(201);                       // 3
  const cuerpo = await res.json();
  expect(cuerpo.tarifa).toBe(54);

  const guardado = await consultarPedido(cuerpo.id);  // 4
  expect(guardado.estado).toBe("REGISTRADO");

  await eliminarPedido(cuerpo.id);                    // 5
});
```

1. **La dirección llega del despliegue**
   La prueba corre contra la versión que se acaba de desplegar.

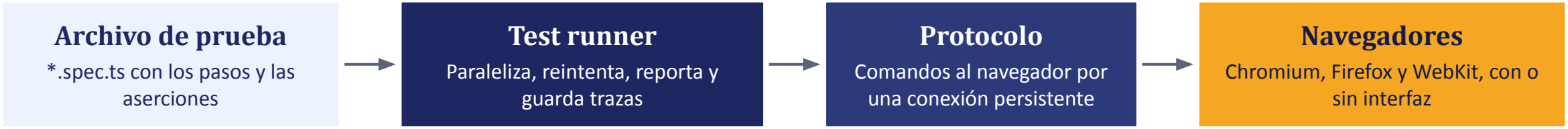
2. **Datos propios con referencia única**
   Dos corridas en paralelo no se pisan.

3. **El estado HTTP confirma que llegó**
   Y el cuerpo, que la respuesta tiene lo esperado.

4. **El efecto persistido confirma el recorrido**
   La respuesta puede ser correcta sin haber guardado nada.

5. **La prueba limpia lo que creó**
   El ambiente queda igual para la corrida siguiente.

# Cómo funciona Playwright




*Navegador → contexto aislado por prueba (cookies y almacenamiento propios) → página*

**1** **Un contexto nuevo por prueba**
Aislamiento equivalente a un perfil limpio, en milisegundos y sin reiniciar el navegador.

**2** **Espera automática antes de cada acción**
Verifica que el elemento esté visible, habilitado y estable. Reemplaza al sleep fijo.

**3** **Localizadores por rol, texto accesible o test id**
getByRole y getByTestId: la selección estable de la lámina anterior.

**4** **Aserciones que reintentan hasta un timeout**
expect(locator).toBeVisible() espera la condición; no compara una sola vez.

**El trace viewer guarda DOM, red, consola y captura de cada paso. En CI se sube como artefacto y permite revisar una falla sin reproducirla.**

# Una prueba end to end con Playwright

TESTS/COMPRA.SPEC.TS

```
import { test, expect } from "@playwright/test";

test("compra de dos entradas", async ({ page, request }) => {
  const r = await request.post("/api/test/eventos", {       // 1
    data: { nombre: `e2e-${Date.now()}`, aforo: 10 } });
  const { id } = await r.json();

  await page.goto(`/eventos/${id}`);                        // 2
  await page.getByTestId("cantidad").fill("2");
  await page.getByRole("button", { name: "Pagar" }).click(); // 3
  await expect(page.getByText("Compra confirmada"))         // 4
    .toBeVisible();
});
```

PLAYWRIGHT.CONFIG.TS · EN CI: npx playwright test --shard=1/4

```
use: { baseURL: process.env.URL_APP, trace: "on-first-retry" },  // 2
retries: process.env.CI ? 2 : 0,                                 // 5
```

**1** **Datos propios creados por API**

Rápido, sin pasar por la interfaz para preparar el caso.

**2** **La dirección llega del despliegue**

baseURL toma la URL que publicó el job anterior.

**3** **Localizadores estables**

Test id y rol accesible; no dependen del diseño.

**4** **Aserción sobre el resultado de negocio**

Reintenta hasta que el texto aparece o vence el timeout.

**5** **Reintenta, sino dato por defecto**

El reporte lo separa y entra a la política de cuarentena.

2

# Pruebas Integración sobre el ambiente desplegado

Qué se verifica después de cada despliegue y qué evidencia decide la promoción

# La secuencia de pruebas a lo largo del pipeline

| Integración continua | Publicar | Certificación | Promover | Producción |
|---|---|---|---|---|
| Unitarias, integración acotada y verificación de contratos. Sin ambiente. | Imagen construida una vez e identificada por su digest. | Digest desplegado, humo, end to end críticas y prueba de carga. | El mismo digest pasa al ambiente protegido con aprobación. | Prueba de humo, ventana de observación y verificación sintética. |

*antes de publicar: segundos a minutos, en cada commit*

*después de desplegar: contra una dirección real, con datos propios*

## Lo que corre antes de publicar decide si existe el artefacto

Una falla ahí no deja imagen en el registro: no hay nada que desplegar.

**El mismo digest recorre todas las etapas. Una prueba en certificación es evidencia para producción solo si lo que se promueve es exactamente lo que se probó.**

# Anatomía de una prueba de humo

SCRIPTS/HUMO.SH · CORRE DESPUÉS DE CADA DESPLIEGUE

```
set -euo pipefail
URL=$1; VERSION_ESPERADA=$2

curl -fsS --retry 5 --retry-delay 3 "$URL/salud"          # 1
v=$(curl -fsS "$URL/version" | jq -r .version)
[ "$v" = "$VERSION_ESPERADA" ] || exit 1                  # 2
curl -fsS "$URL/salud/dependencias"                       # 3
curl -fsS "$URL/eventos?limite=1" > /dev/null             # 4
```

### Sí verifica

Arranque, versión, dependencias y una lectura crítica. Menos de un minuto.

### No verifica

Reglas de negocio ni escrituras con efecto real. Eso es end to end.

**1** **Espera el arranque con reintentos**

Cinco intentos cada tres segundos; después, falla.

**2** **Confirma que corre la versión desplegada**

El balanceador puede seguir enviando tráfico a la anterior.

**3** **Alcanza sus dependencias**

Prueba la red, las credenciales y la configuración del ambiente.

**4** **Una lectura crítica sin efectos**

Se puede repetir en producción sin dejar datos.

# Prueba de humo y verificación sintética

### Prueba de humo

Pocas verificaciones, segundos, justo después de desplegar: el servicio arrancó, responde y alcanza sus dependencias. Si falla, dispara la reversión de la sesión 6.

### Verificación sintética

Un recorrido crítico ejecutado cada pocos minutos contra producción, haya o no despliegue. Detecta un certificado vencido o una dependencia externa caída.

1

### La verificación sintética usa datos marcados como de prueba

No cobra, no notifica y se excluye de las métricas de negocio.

2

### Su resultado es una señal de observabilidad

Entra a la ventana de observación y al indicador del objetivo de servicio del Capítulo 4.

# Pruebas de rendimiento: cuatro preguntas distintas

| Tipo | Pregunta | Perfil de carga | Cuándo corre |
|---|---|---|---|
| **Carga** | ¿Cumple el SLO con el tráfico esperado? | Sostenida al nivel esperado, 15 a 30 min | Antes de promover cambios relevantes |
| **Estrés** | ¿Dónde se rompe y cómo falla? | Creciente hasta degradar | Periódica y antes de eventos |
| **Pico** | ¿Soporta un salto súbito? | De la base a 10 veces en segundos | Antes de un evento conocido: preventa, campaña |
| **Resistencia** | ¿Se degrada con el tiempo? | Moderada durante horas | Programada: fugas de memoria y conexiones |

1

### Se mide en percentiles

p95 = el 95% de las peticiones respondió por debajo de ese tiempo. El promedio oculta la cola lenta.

2

### Cuatro señales a la vez

Latencia, throughput, tasa de error y saturación de CPU, memoria y conexiones.

3

### El ambiente se parece a producción en tamaño

Un resultado sobre una instancia más pequeña no se puede extrapolar.

# El rendimiento como compuerta del pipeline

PRUEBAS/CARGA.JS · k6

```
import http from "k6/http";
import { check } from "k6";

export const options = {
  stages: [                                        // 1
    { duration: "2m", target: 50 },
    { duration: "5m", target: 50 },
    { duration: "1m", target: 0 },
  ],
  thresholds: {                                    // 2
    http_req_duration: ["p(95)<400"],
    http_req_failed:   ["rate<0.01"],
  },
};

export default function () {
  const r = http.get(`${__ENV.URL_API}/eventos`);  // 3
  check(r, { "status 200": (x) => x.status === 200 });
}
```

1. **Perfil declarado en el repositorio**
   Rampa, meseta y bajada: 50 usuarios virtuales.

2. **Umbrales derivados del SLO**
   Si se incumplen, el proceso termina con error y el job falla.

3. **Corre contra certificación**
   La URL llega del despliegue, como las end to end.

4. **Nunca en cada commit**
   Ocho minutos y costo de infraestructura: antes de promover o programada.

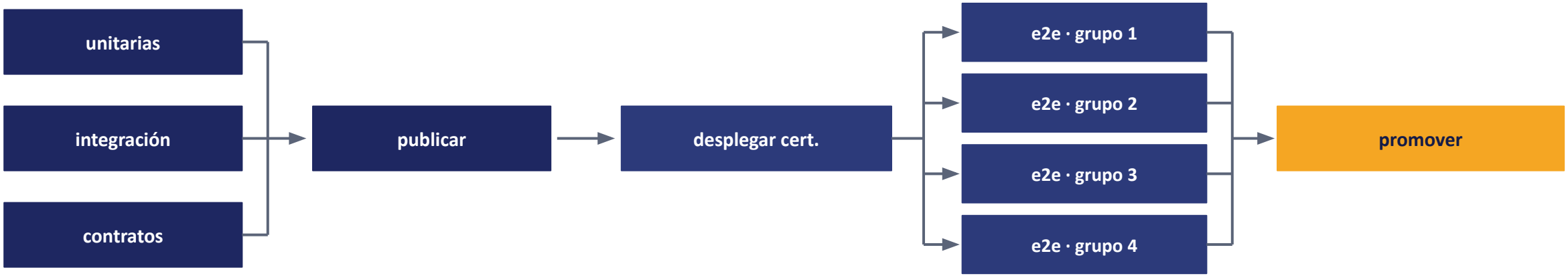
5. **Se compara contra la línea base**
   Una regresión de 20% es señal aunque cumpla el umbral.

3

# Orquestación de pruebas en el pipeline

Orden, paralelismo, reportes y la política que decide qué detiene la promoción

# Orden y paralelismo




1

### Lo rápido y lo que más falla corre primero

Un error de unitaria se conoce en segundos y no consume el tiempo de las etapas siguientes.

2

### Lo independiente corre en paralelo

Unitarias, integración y contratos no se esperan entre sí.

3

### Las end to end se reparten entre ejecutores

Cuatro grupos bajan 30 minutos a unos 8. El costo pasa a minutos de cómputo.

# La secuencia completa en el pipeline

GITHUB ACTIONS — PRUEBAS POR ETAPA

```
jobs:
  unitarias:    { ... }
  integracion: { ... }
  publicar:
    needs: [unitarias, integracion]          # 1
    outputs: { digest: ${{ steps.push.outputs.digest }} }
  desplegar-cert:
    needs: publicar
    outputs: { url: ${{ steps.deploy.outputs.url }} }  # 2
  e2e:
    needs: desplegar-cert
    strategy:
      matrix: { grupo: [1, 2, 3, 4] }        # 3
    steps:
      - run: npm run test:e2e -- --grupo ${{ matrix.grupo }}
        env:
          URL_API: ${{ needs.desplegar-cert.outputs.url }}
      - uses: actions/upload-artifact@v4     # 4
        if: always()
        with: { name: e2e-${{ matrix.grupo }}, path: reportes/ }
  promover:
    needs: [publicar, e2e]
    environment: produccion                  # 5
```

**1** **Publicar espera las compuertas de CI**
Si una falla, no existe imagen nueva.

**2** **El despliegue expone su dirección**
Las end to end la reciben como variable.

**3** **Cuatro ejecutores en paralelo**
Cada grupo corre una parte de la suite.

**4** **El reporte se sube aunque falle**
if: always() conserva la evidencia del rojo.

**5** **Promover usa el ambiente protegido**
Mismo digest y la aprobación de la sesión 6.

# Política de compuertas para pruebas

| Resultado | Efecto | Dónde actúa |
|---|---|---|
| **Falla una prueba unitaria o de integración** | **Detiene** | La publicación del artefacto |
| **Cobertura del código nuevo bajo el umbral** | **Detiene** | La publicación, vía quality gate |
| **Falla un recorrido end to end crítico** | **Detiene** | La promoción a producción |
| **Latencia p95 o tasa de error sobre el umbral en la prueba de carga** | **Detiene** | La promoción a producción |
| **Falla una prueba en cuarentena** | **Informa** | Registro con responsable y vencimiento |
| **Falla la prueba de humo en producción** | **Revierte** | Rollback: vuelve el digest anterior sin aprobación |

# Puntos clave

1. **El contrato protege a los consumidores sin ambiente compartido**
   El proveedor sabe qué campo no puede cambiar antes de desplegar.

2. **La end to end se reserva para recorridos críticos que cruzan servicios**
   Su costo en minutos se justifica con el costo de su falla en producción.

3. **Cada prueba tiene una etapa: antes de publicar o contra el ambiente desplegado**
   Humo, end to end y carga reciben la dirección real del despliegue.

4. **El mismo digest recorre todas las pruebas**
   La evidencia de certificación vale para producción porque el artefacto es el mismo.

5. **La política de compuertas se escribe antes de la primera falla**
   Qué detiene, qué informa y qué revierte queda en el repositorio.